

Bachelor's Thesis Proposal

Instrumenting Application Code for Personal Data Audit Logging

Assessor: Dr. Hoang Nguyen
Co-assessor: To be decided

1 Motivation

Data protection regulations such as the GDPR [3] require data controllers not only to process personal data lawfully but also to be able to *demonstrate* that they do so (Art. 5(2)). Moreover, data subjects have the right to obtain information about how their personal data is processed (Art. 15). Both obligations presuppose a reliable record of *which* action was performed on *whose* personal data, *by whom*, *when*, and *for what purpose*. Such records are also the input to approaches that check compliance after the fact, e.g., by monitoring logs against formalized GDPR provisions [1, 2], and they are a prerequisite for information accountability [6].

In practice, such logs are written by hand: developers insert logging statements wherever they believe personal data is touched. This is tedious, error-prone, and easily falls out of sync with the code. A more systematic approach is to *instrument* the application, i.e., to automatically weave the logging into the code at every point where an action on personal data is invoked. Since the instrumentation observes every such action, the resulting logs are complete by construction with respect to the instrumented actions.

2 Objective

The objective of this thesis is to develop an approach that instruments application code such that whenever an action on personal data is invoked (e.g., create, read, update, delete, or transfer to a third party), a structured log entry is inserted automatically.

Within the scope of the thesis, we restrict the approach to web applications written in a single language and framework (e.g., Python with Flask [4]). We assume that the personal data in the application, together with its data subjects, is identified by an existing code scanning approach. The instrumentation should require no manual changes to the business logic or only minimal ones.

3 Tasks

1. Study the existing privacy requirements (e.g., the GDPR) on records of processing and review techniques for program instrumentation (e.g., aspect-oriented programming [5]), as well as

- existing approaches to audit logging.
2. Define a log schema for actions on personal data and a specification that maps application code (e.g., functions, HTTP routes, or data operations) to the actions on personal data that they perform.
 3. Design an instrumentation approach that, given this specification, automatically inserts logging at the corresponding points in the application code.
 4. Implement the approach for the chosen language and framework, persisting the logs in a store that can be queried, e.g., by data subjects or auditors.
 5. Document the design, implementation, and evaluation in the final report.

4 Deliverables

The following deliverables are due at the end of the project:

Final report Please have a look at the submission requirements in the following section. The final report should consist of an introduction, a (theoretical) background section, sections describing the work packages and their outcomes, and a conclusion. The report must be written in English.

Source code Complete, documented source code.

Presentation At the end of the thesis, a 20-minute presentation must be given in a thesis defense seminar. It should give an overview and discuss the most important highlights of the work.

References

- [1] Emma Arfelt, David A. Basin, and Søren Debois. Monitoring the GDPR. In *Computer Security — ESORICS 2019*, volume 11735 of *Lecture Notes in Computer Science*, pages 681–699. Springer, 2019.
- [2] David A. Basin, Felix Klaedtke, Samuel Müller, and Eugen Zălinescu. Monitoring metric first-order temporal properties. *J. ACM*, 62(2):15:1–15:45, 2015.
- [3] European Parliament and Council. Regulation (EU) 2016/679, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [4] Flask. Flask Web Framework, 2023.
- [5] Gregor Kiczales, John Lamping, Anurag Mendhekar, Chris Maeda, Cristina Videira Lopes, Jean-Marc Loingtier, and John Irwin. Aspect-oriented programming. In *ECOOP’97 — Object-Oriented Programming*, volume 1241 of *Lecture Notes in Computer Science*, pages 220–242. Springer, 1997.
- [6] Daniel J. Weitzner, Harold Abelson, Tim Berners-Lee, Joan Feigenbaum, James Hendler, and Gerald Jay Sussman. Information accountability. *Commun. ACM*, 51(6):82–87, June 2008.